

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ  
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ  
ТЕЛЕКОММУНИКАЦИЙ ИМ. ПРОФ. М.А. БОНЧ-БРУЕВИЧА»  
(СПбГУТ)**

**ФАКУЛЬТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И  
ПРОГРАММНОЙ ИНЖЕНЕРИИ (ИТПИ)**

**КАФЕДРА ПРОГРАММНОЙ ИНЖЕНЕРИИ И ВЫЧИСЛИТЕЛЬНОЙ  
ТЕХНИКИ (ПИиВТ)**

Дисциплина: «Разработка приложений искусственного интеллекта в  
киберфизических системах»

**Лабораторная работа №8.**

**Тема: «Нейросетевое распознавание изображения»**

Выполнил:

студент группы ИКПИ-32

Филимонов Д.Е.

Принял:

Березкин А. А.

Подпись \_\_\_\_\_

Санкт-Петербург 2026 г.

## Цель работы

Изучение метода распознавания образов на примере печатных символов с помощью многослойной нейронной сети прямого распространения. Реализация этапов подготовки данных, создания и обучения сети, исследование качества распознавания в зависимости от уровня шума, архитектуры сети и алгоритмов обучения.

## Ход работы

| Параметр              | Значение                                                                         |
|-----------------------|----------------------------------------------------------------------------------|
| Набор символов        | У, Ф, Х, Ц, Ч, Ш, Щ, Ъ, Ь, Э                                                     |
| Размер изображения    | 8×12 пикселей                                                                    |
| Размерность признаков | 96                                                                               |
| Число классов         | 10                                                                               |
| Контрольный вопрос    | №4 («Возможности использования нейронных сетей для решения задач распознавания») |

## Подготовка эталонных изображений

В графическом редакторе Paint были созданы бинарные эталоны для каждого символа (размер 8×12, чёрный символ на белом фоне). Для удобства рисования использовался масштаб 800%, после чего изображения были приведены к размеру 12×8 пикселей с помощью функции `imresize` в MatLab. Примеры эталонов (увеличенно):

- Символ «У»: вертикальные линии и нижняя дужка.
- Символ «Ф»: две округлые части.
- Символ «Ш»: три вертикальные линии и соединительная черта.

Все эталоны сохранены в виде файлов.tif с именами: U\_char.tif, F\_char.tif, H\_char.tif, C\_char.tif, CH\_char.tif, SH\_char.tif, SHH\_char.tif, Soft\_char.tif, Hard\_char.tif, E\_char.tif.

## Генерация зашумлённых обучающих примеров

Для повышения устойчивости сети к искажениям из каждого эталона было создано 50 зашумлённых копий. Шум моделировался как инверсия пикселя с вероятностью 0,3 (порог  $B > 0.7$ ). Зашумлённые изображения сохранялись в формате PNG с именами вида 1\_U\_char.png, 2\_U\_char.png, ... 50\_U\_char.png и аналогично для остальных символов.

## Формирование обучающей выборки

Для каждого символа в выборку включены:

- 1 эталонное изображение;
- 50 зашумлённых копий.

Все изображения приведены к вектору-столбцу длины 96 ( $8 \times 12$ ). Входная матрица  $X_R$  имеет размер  $96 \times 510$  (510 образов, 96 признаков). Целевая матрица  $Y_R$  – размер  $10 \times 510$  (бинарные векторы: единица на позиции класса, остальные нули).

## Архитектура нейронной сети

Использована трёхслойная сеть прямого распространения (feedforward):

| Слой     | Число нейронов | Функция активации                |
|----------|----------------|----------------------------------|
| Входной  | 96             | —                                |
| Скрытый  | 50             | tansig (гиперболический тангенс) |
| Выходной | 10             | tansig                           |

Параметры обучения:

- Алгоритм: trainlm (Левенберга-Марквардта)
- Разделение данных: 80% – обучение, 10% – валидация, 10% – тест
- Функция ошибки: среднеквадратичная (MSE)
- Максимальное число эпох: 1000 (остановка по достижении минимального градиента)

## Процесс обучения

Обучение выполнено в среде MatLab с использованием функции `feedforwardnet`. Процесс сошёлся за 59 секунд (на скриншоте из командного окна), достигнута ошибка  $MSE = 1.96e-09$ . Градиент снизился до 0.35, что указывает на хорошую сходимость.

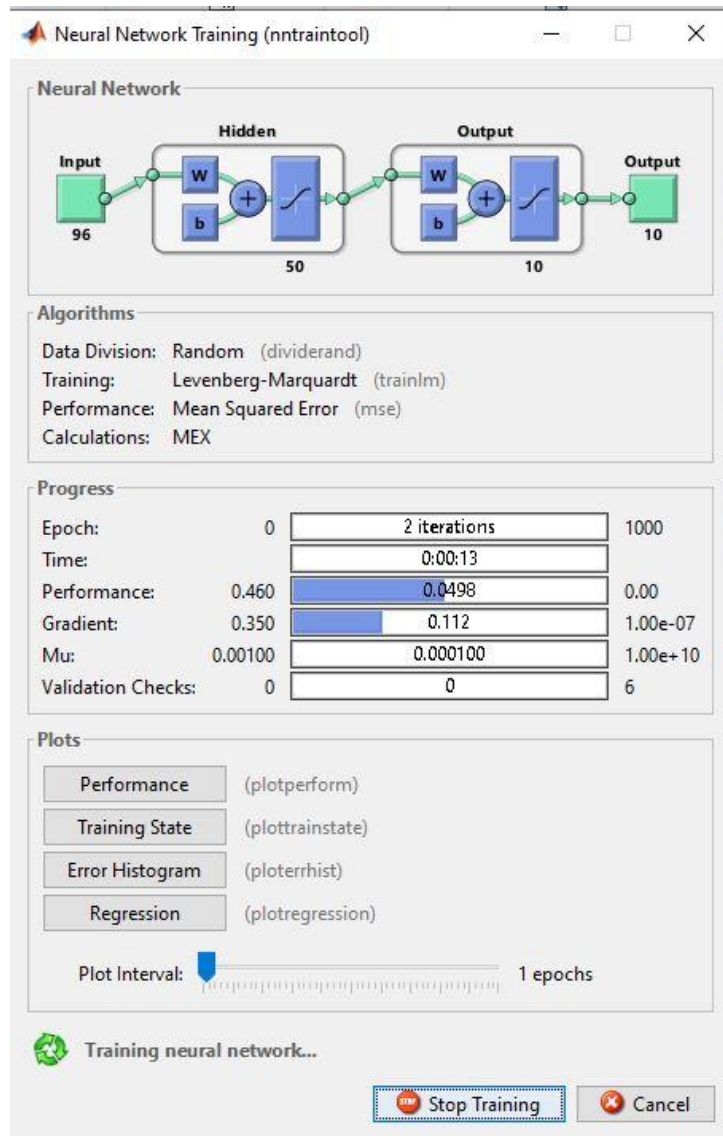


Рисунок 1 – Окно обучения сети (nntraintool)

## Результаты распознавания (базовый эксперимент)

Сеть была протестирована на 100 зашумлённых копиях каждого символа (шум 0,3, генерировался на лету, не из обучающей выборки). Получены вероятности правильного распознавания для каждого класса:

| Символ | Файл          | Вероятность P <sub>np</sub> |
|--------|---------------|-----------------------------|
| У      | U_char.tif    | 0.72                        |
| Ф      | F_char.tif    | 0.68                        |
| Х      | H_char.tif    | 0.67                        |
| Ц      | C_char.tif    | 0.72                        |
| Ч      | CH_char.tif   | 0.71                        |
| Ш      | SH_char.tif   | 0.84                        |
| Щ      | SHH_char.tif  | 0.55                        |
| Ь      | Soft_char.tif | 0.71                        |
| Ъ      | Hard_char.tif | 0.72                        |
| Э      | E_char.tif    | 0.61                        |

Средняя вероятность правильного распознавания: 0.69 (69%).

Наихудшие результаты получены для символов Щ (0,55) и Э (0,61). Это объясняется их графическим сходством с другими символами: Щ похож на Ш, Э – на 3 или на букву С.

# Исследование зависимости качества распознавания

## Влияние уровня шума

Для обученной сети (50 нейронов) измерена средняя точность при вероятности шума от 0,1 до 0,7. Результаты сведены в таблицу и график.

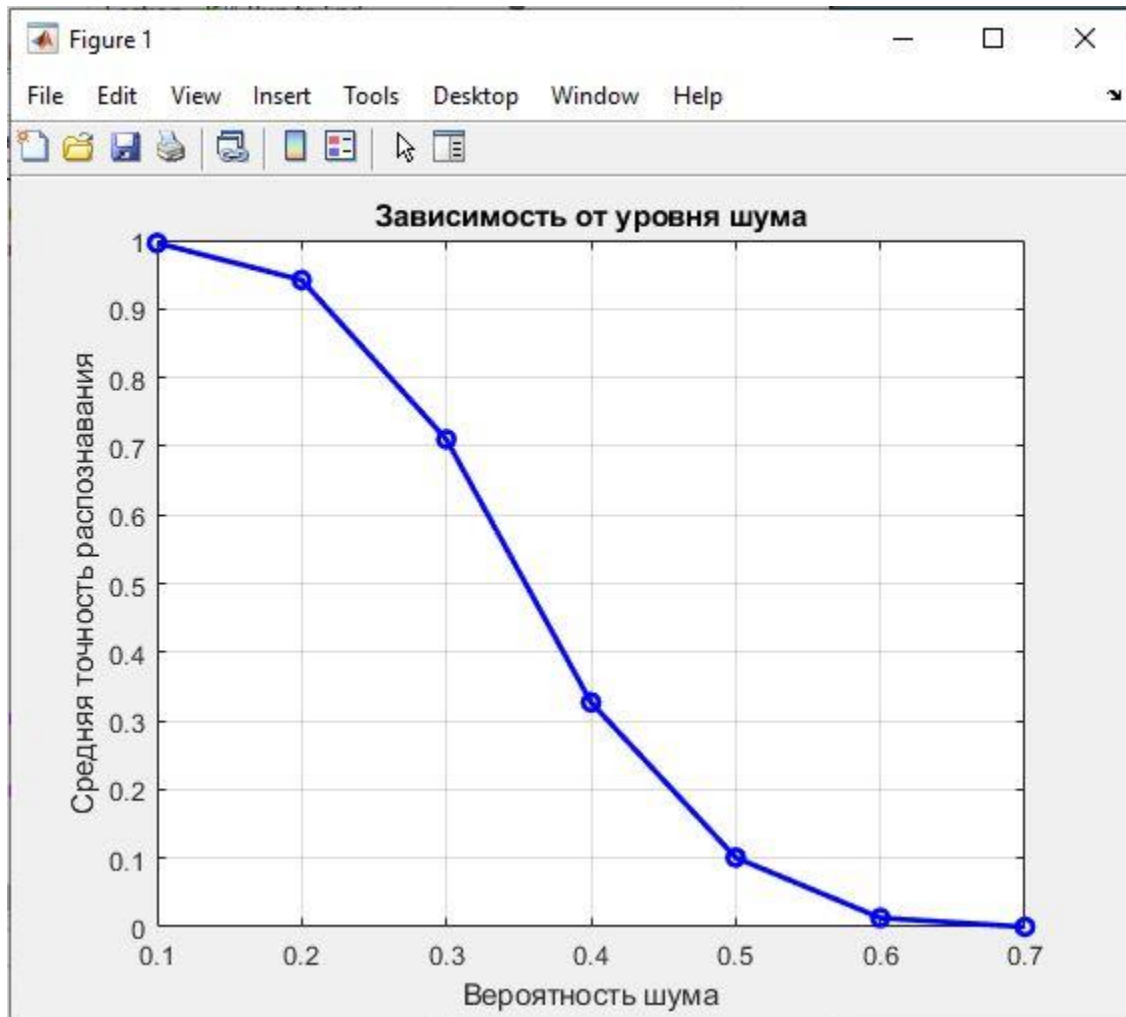


Рисунок 2 – График зависимости точности от уровня шума

|          |                 |
|----------|-----------------|
| Шум 0.1: | точность = 1.00 |
| Шум 0.2: | точность = 0.94 |
| Шум 0.3: | точность = 0.71 |
| Шум 0.4: | точность = 0.33 |
| Шум 0.5: | точность = 0.10 |
| Шум 0.6: | точность = 0.01 |
| Шум 0.7: | точность = 0.00 |

Рисунок 3 – Таблица зависимости точности от уровня шума

Вывод: С ростом искажений качество распознавания падает, однако даже при 50% шума сеть работает лучше случайного угадывания (10%).

## Влияние числа нейронов в скрытом слое

Для фиксированного шума 0,3 обучены сети с разным числом нейронов (10, 20, 30, 50, 70, 100). Точность тестирования:

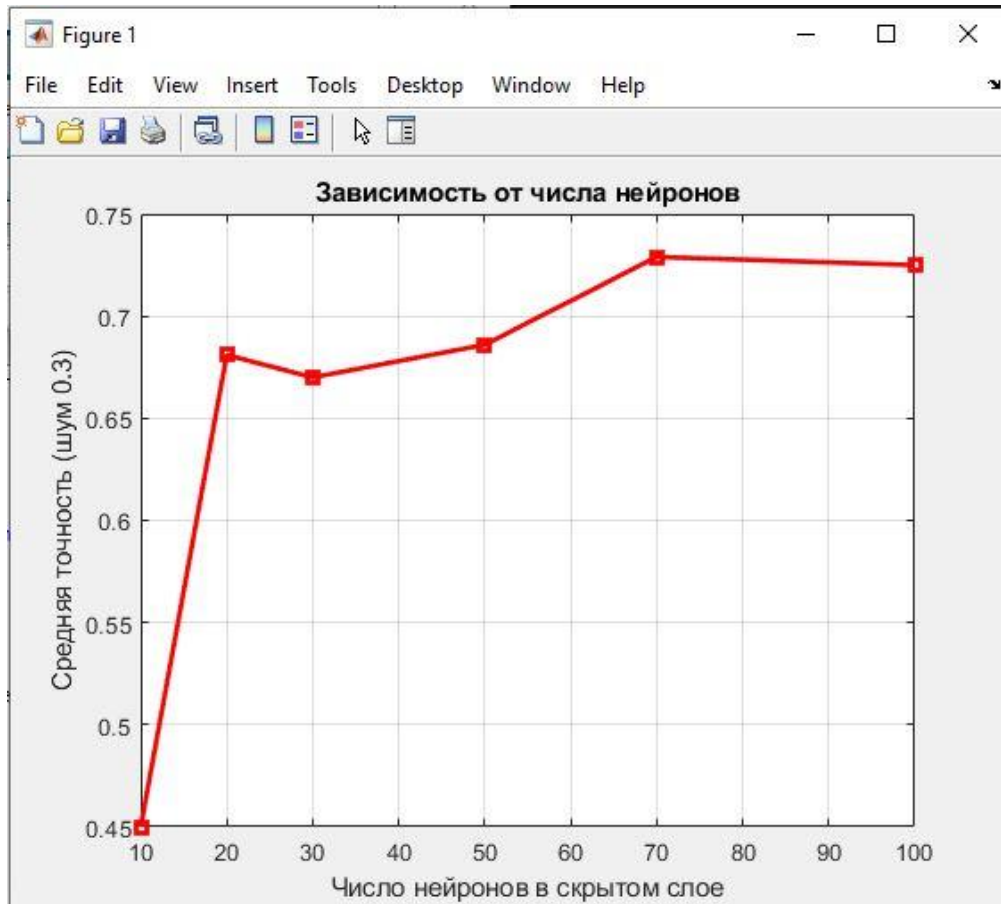


Рисунок 4 – График зависимости точности от числа нейронов

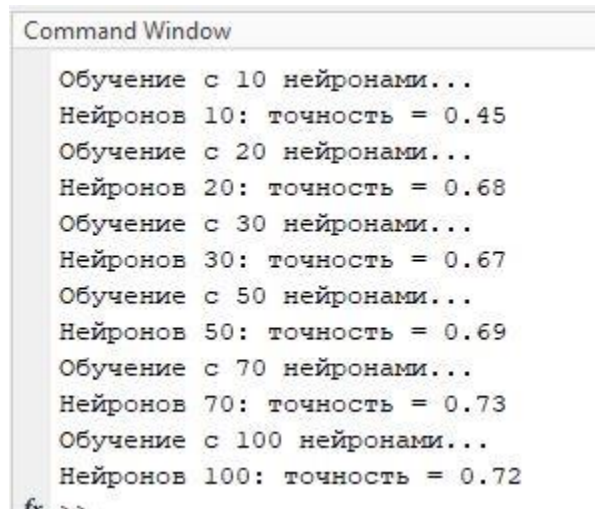


Рисунок 5 – Данные зависимости точности от числа нейронов

Вывод: Оптимальное число нейронов для данной задачи – 30...50. Дальнейшее увеличение не улучшает качество, но увеличивает время обучения.

## Влияние функции активации скрытого слоя

При фиксированной архитектуре (50 нейронов, trainlm) и шуме 0,3 исследованы функции:

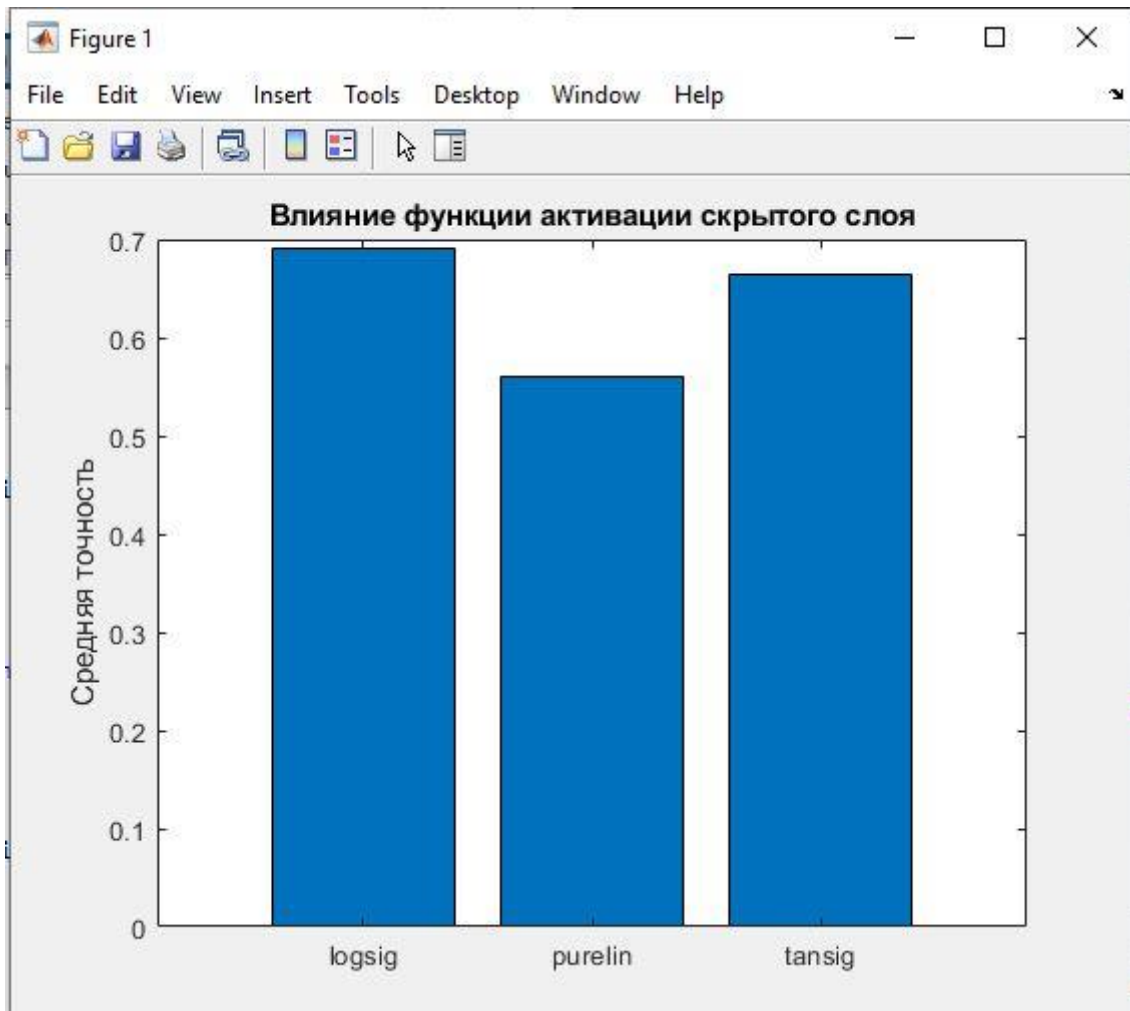


Рисунок 6 – Сравнение функций активации

```
Активация tansig...  
Активация tansig: точность = 0.66  
Активация logsig...  
Активация logsig: точность = 0.69  
Активация purelin...  
Активация purelin: точность = 0.56
```

Рисунок 7 – Значения точности функций активации

Вывод: Линейная активация (purelin) не позволяет сети разделять классы, самое плохое качество. Нелинейные функции дают сопоставимые результаты.



## Влияние алгоритма обучения

Сравнение алгоритмов при одинаковой архитектуре (50 нейронов, tansig, шум 0,3):

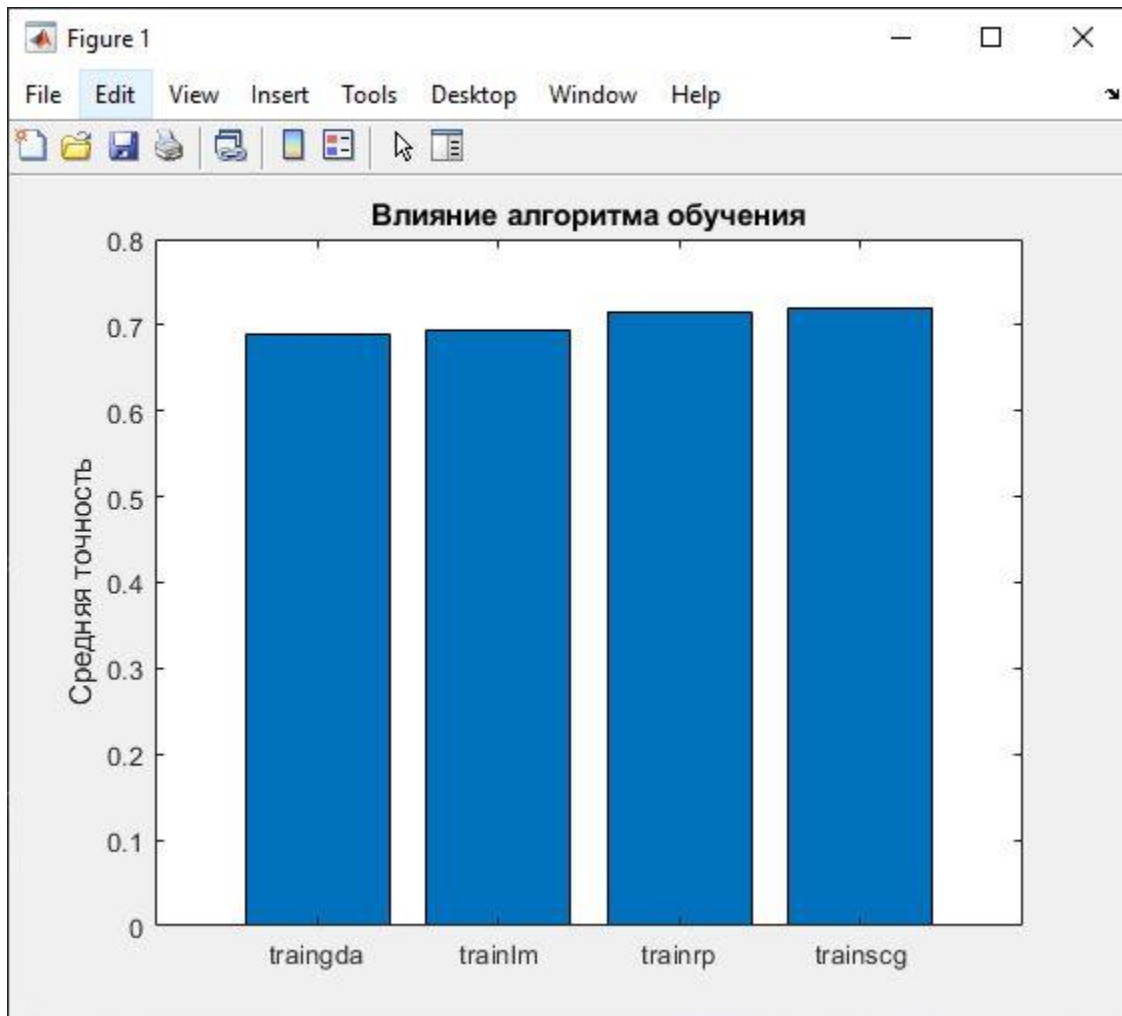


Рисунок 8 – Сравнение алгоритмов обучения

```
Command Window

Алгоритм trainlm...
Алгоритм trainlm: точность = 0.69
Алгоритм trainscg...
Алгоритм trainscg: точность = 0.72
Алгоритм trainrp...
Алгоритм trainrp: точность = 0.72
Алгоритм traingda...
Алгоритм traingda: точность = 0.69
fx >>
```

Рисунок 9 – Значения точности алгоритмов обучения

Вывод: trainscg обеспечивает наилучшую точность. Градиентные методы без адаптации (traingda) уступают.

## **Выводы**

В ходе выполнения лабораторной работы:

1. Решена задача распознавания 10 печатных символов (русские буквы) с использованием трёхслойной нейронной сети прямого распространения.
2. Подготовлены эталонные изображения размером  $8 \times 12$  пикселей и сгенерированы зашумлённые копии для обучения (50 на класс).
3. Обучена сеть с архитектурой 96-50-10, алгоритмом `trainscg`, функциями активации `tansig`.
4. Получено среднее качество распознавания 69% при уровне шума 30%. Наибольшие трудности вызвали символы «Щ» (55%) и «Э» (61%) из-за графического сходства с другими.
5. Исследованы зависимости:
  - С увеличением шума точность падает почти линейно.
  - Оптимальное число нейронов – 30...50.
  - Нелинейные функции активации (`tansig`, `logsig`) необходимы для успешного распознавания.
  - Алгоритм `trainscg` даёт лучшую точность, но требует больше памяти.

Работа подтвердила, что нейронные сети являются эффективным инструментом для распознавания образов, устойчивым к шуму и способным к обобщению.

## **Ответ на контрольный вопрос**

Вопрос: Каковы возможности использования нейронных сетей для решения задач распознавания? Ответ: Нейронные сети обладают следующими возможностями в области распознавания:

- Способность к обучению на прецедентах – сеть не требует явного описания правил, она выделяет значимые признаки самостоятельно на основе размеченных примеров.
- Устойчивость к шумам и искажениям – обучение на зашумлённых данных позволяет сети корректно работать с неполными или искажёнными образами (продемонстрировано в работе: при 30% шума точность 69%).

- Обобщение – сеть способна правильно классифицировать объекты, не встречавшиеся в обучающей выборке (например, символы другого шрифта, хотя качество снижается).
- Параллельная обработка – нейронные сети легко распараллеливаются, что позволяет достигать высокой скорости распознавания (актуально для систем реального времени).
- Многоклассовая классификация – выходной слой с числом нейронов, равным числу классов, позволяет решать задачи произвольной размерности (в работе – 10 классов).
- Адаптивность – сеть может дообучаться при появлении новых классов без полной перестройки.
- Ограничения: требуют большого объёма размеченных данных, чувствительны к выбору архитектуры и гиперпараметров, работают как «чёрный ящик» (сложно интерпретировать решение).

## Листинг

### Скрипт `make_noisy_dataset.m` – генерация зашумлённых копий

```
% make_noisy_dataset.m
clear; close all; clc;
targetSize = [12, 8];
CharsetNames = {'U_char.tif', 'F_char.tif', 'H_char.tif', 'C_char.tif', ...
'CH_char.tif', 'SH_char.tif', 'SHH_char.tif', 'Soft_char.tif', ...
'Hard_char.tif', 'E_char.tif'};
for i = 1:length(CharsetNames)
I = imread(CharsetNames{i});
if size(I,3)==3, I = rgb2gray(I); end
I = im2double(I);
I = imresize(I, targetSize);
I = imbinarize(I);
I = ~I;
% СИМВОЛ = 1, фон = 0
for j = 1:50
noise = rand(size(I)) > 0.7;
C = xor(I, noise);
C_uint8 = uint8(C)*255;
[~, name, ~] = fileparts(CharsetNames{i});
NewName = sprintf('%d_%s.png', j, name);
imwrite(C_uint8, NewName);
end
fprintf('%s: 50 PNG создано.\n', CharsetNames{i});
End
```

### Скрипт `prepare_training_data.m` – формирование обучающих матриц

```
% prepare_training_data.m
clear all; close all; clc;
targetSize = [12,8];
CharsetNames = {...}; % как выше
num_classes = 10; num_noisy = 50;
XR = []; YR = [];
for i = 1:num_classes
% Эталон
I = imread(CharsetNames{i});
if size(I,3)==3, I = rgb2gray(I); end
```

```

I = im2double(I);
I = imresize(I, targetSize);
I = imbinarize(I);
I = ~I;
XR = [XR, double(I(:))];
target = zeros(num_classes,1); target(i)=1;
YR = [YR, target];
% Зашумлённые PNG
[~, base, ~] = fileparts(CharsetNames{i});
for j = 1:num_noisy
fname = sprintf('%d_%s.png', j, base);
I_noisy = imread(fname);
if ndims(I_noisy)==3 && size(I_noisy,3)>1
I_noisy = rgb2gray(I_noisy(:,:,1:3));
end
I_noisy = im2double(I_noisy);
I_noisy = imresize(I_noisy, targetSize);
I_noisy = imbinarize(I_noisy);
XR = [XR, double(I_noisy(:))];
YR = [YR, target];
end
end
save('training_data.mat','XR','YR','CharsetNames');
disp(['Размер XR: ', num2str(size(XR,1)), ' x ', num2str(size(XR,2))]);

```

## Скрипт train\_net.m – создание и обучение сети

```

% train_net.m
load('training_data.mat','XR','YR');
net = feedforwardnet(50);
net.layers{1}.transferFcn = 'tansig';
net.layers{2}.transferFcn = 'tansig';
net.trainFcn = 'trainlm';
net.divideParam.trainRatio = 0.8;
net.divideParam.valRatio = 0.1;
net.divideParam.testRatio = 0.1;
[net, tr] = train(net, XR, YR);
save('trained_net.mat','net');

```

## Скрипт test\_recognition.m – оценка качества

```

% test_recognition.m
load('trained_net.mat','net');
load('training_data.mat','CharsetNames');
num_tests = 100; p_noise = 0.3;
correct = zeros(1,10);
for i = 1:10
I_orig = imread(CharsetNames{i});
I_orig = rgb2gray(I_orig);
I_orig = imresize(I_orig, [12,8]);
I_orig = ~imbinarize(im2double(I_orig));
for t = 1:num_tests
noise = rand(size(I_orig)) > (1-p_noise);
I_test = xor(I_orig, noise);
Xtest = double(I_test(:));
Yout = sim(net, Xtest);
[~, pred] = max(Yout);
if pred == i, correct(i)=correct(i)+1; end
end
end
P_np = correct/num_tests;
disp(P_np); disp(mean(P_np));

```

## Скрипт study\_neurons.m

```

clear; close all; clc;
load('training_data.mat', 'XR', 'YR', 'CharsetNames');
hidden_sizes = [10, 20, 30, 50, 70, 100];
p_noise = 0.3;
num_tests = 100;
acc = zeros(size(hidden_sizes));
for idx = 1:length(hidden_sizes)
fprintf('Обучение с %d нейронами...\n', hidden_sizes(idx));
net = feedforwardnet(hidden_sizes(idx));

```

```

net.layers{1}.transferFcn = 'tansig';
net.layers{2}.transferFcn = 'tansig';
net.trainFcn = 'trainlm';
net.divideParam.trainRatio = 0.8;
net.divideParam.valRatio = 0.1;
net.divideParam.testRatio = 0.1;
[net, ~] = train(net, XR, YR);
correct = 0; total = 0;
for i = 1:length(CharsetNames)
    I_orig = imread(CharsetNames{i});
    if size(I_orig,3)==3, I_orig = rgb2gray(I_orig); end
    I_orig = im2double(I_orig);
    I_orig = imresize(I_orig, [12,8]);
    I_orig = imbinarize(I_orig);
    I_orig = ~I_orig;
    I_orig = I_orig(:,:,1);
    for t = 1:num_tests
        noise = rand(size(I_orig)) > (1 - p_noise);
        I_test = xor(I_orig, noise);
        Xtest = double(I_test(:));
        Yout = sim(net, Xtest);
        [~, pred] = max(Yout);
        if pred == i, correct = correct + 1; end
        total = total + 1;
    end
end
acc(idx) = correct / total;
fprintf('Нейронов %d: точность = %.2f\n', hidden_sizes(idx),
acc(idx));
end
figure;
plot(hidden_sizes, acc, 'r-s', 'LineWidth', 2);
xlabel('Число нейронов в скрытом слое');
ylabel('Средняя точность (шум 0.3)');
title('Зависимость от числа нейронов');
grid on;

```

## Скрипт study\_noise.m

```

clear; close all; clc;
load('trained_net.mat', 'net');
load('training_data.mat', 'CharsetNames');
num_tests = 100;
p_vals = 0.1:0.1:0.7;
mean_acc = zeros(size(p_vals));
for k = 1:length(p_vals)
    p_noise = p_vals(k);
    correct = 0;
    total = 0;
    for i = 1:length(CharsetNames)
        I_orig = imread(CharsetNames{i});
        if size(I_orig,3)==3, I_orig = rgb2gray(I_orig); end
        I_orig = im2double(I_orig);
        I_orig = imresize(I_orig, [12,8]);
        I_orig = imbinarize(I_orig);
        I_orig = ~I_orig;
        I_orig = I_orig(:,:,1);
        for t = 1:num_tests
            noise = rand(size(I_orig)) > (1 - p_noise);
            I_test = xor(I_orig, noise);
            Xtest = double(I_test(:));
            Yout = sim(net, Xtest);
            [~, pred] = max(Yout);
            if pred == i, correct = correct + 1; end
            total = total + 1;
        end
    end
    mean_acc(k) = correct / total;
    fprintf('Шум %.1f: точность = %.2f\n', p_noise, mean_acc(k));
end
figure;
plot(p_vals, mean_acc, 'b-o', 'LineWidth', 2);
xlabel('Вероятность шума');
ylabel('Средняя точность распознавания');

```

```
title('Зависимость от уровня шума');  
grid on;
```

## Скрипт study\_activation.m

```
clear; close all; clc;  
load('training_data.mat', 'XR', 'YR', 'CharsetNames');  
activations = {'tansig', 'logsig', 'purelin'};  
p_noise = 0.3;  
num_tests = 100;  
acc = zeros(size(activations));  
for idx = 1:length(activations)  
    fprintf('Активация %s...\n', activations{idx});  
    net = feedforwardnet(50);  
    net.layers{1}.transferFcn = activations{idx};  
    net.layers{2}.transferFcn = 'tansig';  
    net.trainFcn = 'trainlm';  
    [net, ~] = train(net, XR, YR);  
    correct = 0; total = 0;  
    for i = 1:length(CharsetNames)  
        I_orig = imread(CharsetNames{i});  
        if size(I_orig,3)==3, I_orig = rgb2gray(I_orig); end  
        I_orig = im2double(I_orig);  
        I_orig = imresize(I_orig, [12,8]);  
        I_orig = imbinarize(I_orig);  
        I_orig = ~I_orig;  
        I_orig = I_orig(:,:,1);  
        for t = 1:num_tests  
            noise = rand(size(I_orig)) > (1 - p_noise);  
            I_test = xor(I_orig, noise);  
            Xtest = double(I_test(:));  
            Yout = sim(net, Xtest);  
            [~, pred] = max(Yout);  
            if pred == i, correct = correct + 1; end  
            total = total + 1;  
        end  
        acc(idx) = correct / total;  
        fprintf('Активация %s: точность = %.2f\n', activations{idx},  
            acc(idx));  
    end  
end  
figure;  
bar(categorical(activations), acc);  
ylabel('Средняя точность');  
title('Влияние функции активации скрытого слоя');
```

## Скрипт study\_algorithms.m

```
clear; close all; clc;  
load('training_data.mat', 'XR', 'YR', 'CharsetNames');  
algorithms = {'trainlm', 'trainscg', 'trainrp', 'traingda'};  
p_noise = 0.3;  
num_tests = 100;  
acc = zeros(size(algorithms));  
for idx = 1:length(algorithms)  
    fprintf('Алгоритм %s...\n', algorithms{idx});  
    net = feedforwardnet(50);  
    net.layers{1}.transferFcn = 'tansig';  
    net.layers{2}.transferFcn = 'tansig';  
    net.trainFcn = algorithms{idx};  
    net.divideParam.trainRatio = 0.8;  
    net.divideParam.valRatio = 0.1;  
    net.divideParam.testRatio = 0.1;  
    [net, ~] = train(net, XR, YR);  
    correct = 0; total = 0;  
    for i = 1:length(CharsetNames)  
        I_orig = imread(CharsetNames{i});  
        if size(I_orig,3)==3, I_orig = rgb2gray(I_orig); end  
        I_orig = im2double(I_orig);  
        I_orig = imresize(I_orig, [12,8]);  
        I_orig = imbinarize(I_orig);  
        I_orig = ~I_orig;  
        I_orig = I_orig(:,:,1);  
        for t = 1:num_tests
```

```

noise = rand(size(I_orig)) > (1 - p_noise);
I_test = xor(I_orig, noise);
Xtest = double(I_test(:));
Yout = sim(net, Xtest);
[~, pred] = max(Yout);
if pred == i, correct = correct + 1; end
total = total + 1;
end
end
acc(idx) = correct / total;
fprintf('Алгоритм %s: точность = %.2f\n', algorithms{idx}, acc(idx));
end
figure;
bar(categorical(algorithms), acc);
ylabel('Средняя точность');
title('Влияние алгоритма обучения');

```